



UTM
UNIVERSITI TEKNOLOGI MALAYSIA

FACULTY OF COMPUTING

SEMESTER 1 2025/2026

SECP3623-02 DATABASE PROGRAMMING

SECTION 02

TITLE:

LAB ASSIGNMENT 2 (INDIVIDUAL)

LECTURER: DR. SHAMINI A/P RAJA KUMARAN

NAME OF STUDENTS	MATRIC NO.
MUHAMMAD SYAHMI FARIS BIN RUSLI	A23CS0138

PART A - DATA PREPARATION

Question A1: Create a MongoDB database named clinicdb and create the following collections:

- appointments
- loginSessions

The screenshot shows the MongoDB Cloud console. On the left, the 'Create Database' dialog is open. It has a 'Database Name' field with 'clinicdb' entered and a 'Collection Name' field with 'appointments' entered. There is a checkbox for 'Time-Series' which is unchecked. Below it, there is a link to 'Learn More'. At the bottom of the dialog, there is a 'Cancel' button and a 'Create Database' button. On the right, the database 'clinicdb' is shown with two collections: 'appointments' and 'loginSessions'.

Question A2: Insert at least six (6) documents into each collection with varied and realistic data, including different statuses, departments, roles, and fees.

- appointments

Query:

```
> db.appointments.insertMany([
  {
    appointmentId: "A001",
    patientName: "Ali Ahmad",
    doctor: "Dr Lim",
    department: "Cardiology",
    status: "COMPLETED",
    fee: 250,
    appointmentDate: ISODate("2025-01-05")
  },
  {
    appointmentId: "A002",
    patientName: "Siti Aminah",
    doctor: "Dr Wong",
    department: "Dermatology",
    status: "PENDING",
    fee: 120,
    appointmentDate: ISODate("2025-01-10")
  },
  {
    appointmentId: "A003",
    patientName: "John Tan",
    doctor: "Dr Kumar",
    department: "Orthopedics",
    status: "CANCELLED",
    fee: 180,
    appointmentDate: ISODate("2025-01-12")
  },
  {
    appointmentId: "A004",
    patientName: "Nur Aisyah",
    doctor: "Dr Lim",
    department: "Cardiology",
    status: "COMPLETED",
    fee: 300,
    appointmentDate: ISODate("2025-01-15")
  },
  {
    appointmentId: "A005",
    patientName: "Daniel Lee",
    doctor: "Dr Wong",
    department: "Dermatology",
    status: "COMPLETED",
    fee: 90,
    appointmentDate: ISODate("2025-01-18")
  },
  {
    appointmentId: "A006",
    patientName: "Farah Zainal",
    doctor: "Dr Kumar",
    department: "Orthopedics",
    status: "PENDING",
    fee: 200,
    appointmentDate: ISODate("2025-01-20")
  }
])
```

Result:

```
{
  acknowledged: true,
  insertedIds: {
    '0': ObjectId('696c44dd9bf2ded5fc9c0298'),
    '1': ObjectId('696c44dd9bf2ded5fc9c0299'),
    '2': ObjectId('696c44dd9bf2ded5fc9c029a'),
    '3': ObjectId('696c44dd9bf2ded5fc9c029b'),
    '4': ObjectId('696c44dd9bf2ded5fc9c029c'),
    '5': ObjectId('696c44dd9bf2ded5fc9c029d')
  }
}
```

- loginSessions

Query:

```
db.loginSessions.insertMany([
  {
    staffId: "S001",
    staffName: "Amir Hassan",
    role: "Doctor",
    status: "ACTIVE",
    ipAddress: "192.168.1.10",
    loginTime: ISODate("2025-01-21T08:30:00")
  },
  {
    staffId: "S002",
    staffName: "Linda Cho",
    role: "Nurse",
    status: "ACTIVE",
    ipAddress: "192.168.1.11",
    loginTime: ISODate("2025-01-21T09:00:00")
  },
  {
    staffId: "S003",
    staffName: "Rahman Aziz",
    role: "Admin",
    status: "INACTIVE",
    ipAddress: "192.168.1.12",
    loginTime: ISODate("2025-01-20T17:45:00")
  },
  {
    staffId: "S004",
    staffName: "Jessica Tan",
    role: "Doctor",
    status: "ACTIVE",
    ipAddress: "192.168.1.10",
    loginTime: ISODate("2025-01-21T10:15:00")
  },
  {
    staffId: "S005",
    staffName: "Mohd Faiz",
    role: "Nurse",
    status: "INACTIVE",
    ipAddress: "192.168.1.13",
    loginTime: ISODate("2025-01-19T14:20:00")
  },
  {
    staffId: "S006",
    staffName: "Aina Sofea",
    role: "Receptionist",
    status: "ACTIVE",
    ipAddress: "192.168.1.14",
    loginTime: ISODate("2025-01-21T11:00:00")
  }
])
```

Result:

```
{
  acknowledged: true,
  insertedIds: {
    '0': ObjectId('696c46149bf2ded5fc9c029e'),
    '1': ObjectId('696c46149bf2ded5fc9c029f'),
    '2': ObjectId('696c46149bf2ded5fc9c02a0'),
    '3': ObjectId('696c46149bf2ded5fc9c02a1'),
    '4': ObjectId('696c46149bf2ded5fc9c02a2'),
    '5': ObjectId('696c46149bf2ded5fc9c02a3')
  }
}
```

Part B - APPOINTMENT REPORTING

Question B1 - Advanced Query & Sorting

Retrieve appointment records that meet all of the following conditions:

- appointment status is COMPLETED
- fee is between 100 and 300 (inclusive)

Query:

```
db.appointments.find({status: "COMPLETED", fee: {$gte: 100, $lte: 300}})
```

Result:

```
{
  _id: ObjectId('696c44dd9bf2ded5fc9c0298'),
  appointmentId: 'A001',
  patientName: 'Ali Ahmad',
  doctor: 'Dr Lim',
  department: 'Cardiology',
  status: 'COMPLETED',
  fee: 250,
  appointmentDate: 2025-01-05T00:00:00.000Z
}
{
  _id: ObjectId('696c44dd9bf2ded5fc9c029b'),
  appointmentId: 'A004',
  patientName: 'Nur Aisyah',
  doctor: 'Dr Lim',
  department: 'Cardiology',
  status: 'COMPLETED',
  fee: 300,
  appointmentDate: 2025-01-15T00:00:00.000Z
}
```

Display only the following fields:

- appointmentId
- patientName
- department
- fee

Sort the results by:

- department in ascending order (A–Z)
- fee in descending order (highest first)

Query:

```
> db.appointments.find({}, {_id:0, appointmentId:1, department:1, fee:1}).sort({department:1, fee:-1})
```

Result:

```
< {
  appointmentId: 'A004',
  department: 'Cardiology',
  fee: 300
}
{
  appointmentId: 'A001',
  department: 'Cardiology',
  fee: 250
}
{
  appointmentId: 'A002',
  department: 'Dermatology',
  fee: 120
}
{
  appointmentId: 'A005',
  department: 'Dermatology',
  fee: 90
}
{
  appointmentId: 'A006',
  department: 'Orthopedics',
  fee: 200
}
{
  appointmentId: 'A003',
  department: 'Orthopedics',
  fee: 180
}
```

Question B2 - Aggregation Summary

Using MongoDB aggregation, generate a report that shows for each department:

- total number of appointments
- total fees collected

Sort the output by total fees in descending order.

Query:

```
> db.appointments.aggregate([
  {
    $group: {
      _id: "$department",
      totalAppointments: {$sum:1},
      totalFees: {$sum: "$fee"}
    }
  },
  {$sort: {totalFees: -1}}
])
```

Result:

```
{
  _id: 'Cardiology',
  totalAppointments: 2,
  totalFees: 550
}
{
  _id: 'Orthopedics',
  totalAppointments: 2,
  totalFees: 380
}
{
  _id: 'Dermatology',
  totalAppointments: 2,
  totalFees: 210
}
```

PART C - CLINIC SECURITY MONITORING

Question C1 - Active Staff Sessions

Retrieve all ACTIVE staff login sessions and display only:

- staffName
- role
- ipAddress
- loginTime

Sort the results by most recent login time first.

Query:

```
> db.loginSessions.find({}, {_id:0, staffName:1, role:1, ipAddress:1, loginTime:1}).sort({loginTime:-1})
```

Result:

```
< {
  staffName: 'Aina Sofea',
  role: 'Receptionist',
  ipAddress: '192.168.1.14',
  loginTime: 2025-01-21T11:00:00.000Z
}
{
  staffName: 'Jessica Tan',
  role: 'Doctor',
  ipAddress: '192.168.1.10',
  loginTime: 2025-01-21T10:15:00.000Z
}
{
  staffName: 'Linda Cho',
  role: 'Nurse',
  ipAddress: '192.168.1.11',
  loginTime: 2025-01-21T09:00:00.000Z
}
{
  staffName: 'Amir Hassan',
  role: 'Doctor',
  ipAddress: '192.168.1.10',
  loginTime: 2025-01-21T08:30:00.000Z
}
{
  staffName: 'Rahman Aziz',
  role: 'Admin',
  ipAddress: '192.168.1.12',
  loginTime: 2025-01-20T17:45:00.000Z
}
{
  staffName: 'Mohd Faiz',
  role: 'Nurse',
  ipAddress: '192.168.1.13',
  loginTime: 2025-01-19T14:20:00.000Z
}
```


Question C2 - Distinct & Count

Using MongoDB operations:

a) List all distinct IP addresses used by staff with ACTIVE sessions.

Query:

```
> db.loginSessions.distinct(  
  "ipAddress",  
  {status:"ACTIVE"}  
)
```

Result:

```
< [ '192.168.1.10', '192.168.1.11', '192.168.1.14' ]
```

b) Calculate the number of unique staff members who currently have ACTIVE sessions.

Query:

```
db.loginSessions.aggregate([  
  {$match: {status:"ACTIVE"}},  
  {$group: {_id: "$staffId"}},  
  {$count: "uniqueActiveStaff"}  
])
```

Result:

```
{  
  uniqueActiveStaff: 4  
}
```

PART D - INDEXING & DATA MAINTENANCE

Question D1 - Compound Index

Create a compound index on the appointments collection to optimise queries that filter data by:

- department
- status

Query:

```
> db.appointments.createIndex({department:1, status:1})
```

Result:

```
< department_1_status_1
```

Question D2 - Update & Delete Operations

a) Update all appointments with status CANCELLED to ARCHIVED.

Query:

```
> db.appointments.updateMany(  
  {status: "CANCELLED"},  
  {$set: {status: "ARCHIVED"}}  
)
```

Result:

```
{  
  acknowledged: true,  
  insertedId: null,  
  matchedCount: 1,  
  modifiedCount: 1,  
  upsertedCount: 0  
}
```

b) Delete all login session records with status INACTIVE.

Query:

```
> db.loginSessions.deleteMany(  
  {status: "INACTIVE"}  
)
```

Result:

```
< {  
  acknowledged: true,  
  deletedCount: 2  
}
```